Web Architectures

Rimbert Rudisch-Sommer
rimbert.rudisch-sommer@fh-hagenberg.at

Recap: The Canvas Element

- "The canvas element provides scripts with a resolution-dependent bitmap canvas, which can be used for rendering graphs, game graphics, or other visual images on the fly."

- canvas element Attributes
  - width, height and general html-Attributes (id, name, …)
- Example:

```
<canvas id="c" width="200" height="200">
</canvas>
```

- DOM-Function: getContext()
  - Context: Drawing API ("2d" / "webgl")

```
let ctx= document.getElementById("c").getContext("2d");
```

Recap: Canvas 2D Context

- Coordinate system:
  - (0,0) upper left corner
  - y down, x to the right
- Overview API Methods
  - Rectangles
  - Styles (fill, stroke) (supporting gradients)
  - Text
  - Paths
  - Import images / videos / other canvas' content
    - drawImage
  - Transformation
    - rotation, scaling, translation, ...
  - Pixel API
  - toDataURL

Recap: Animation Loop

- requestAnimationFrame()
  - browser calls draw function in optimal way
  - depending on other tasks, visibility, ...

```
function draw() {

  // draw code:
  ...

  // continue animation:
  requestAnimationFrame(draw);
}

// start animation:
requestAnimationFrame(draw);
```

Recap: SVG

- Scalable Vector Graphics
  - describe graphics in XML
  - resolution independent
  - each shape is an object – scriptable / styleable

```
<svg width="400" height="400">
    <rect width="200" height="50" style="fill:#FF0000;stroke-width:2;stroke:#000000" />
</svg>
```

- Shapes: line, rectangle, circle, ellipse, polyline, polygon
- Paths: moveto, lineto, closepath, curves (Bezier, Elliptical Arc), …
- Further: Texts, Groups, Embedded Images, Filters, …

Recap: Asynchronity

- Where is the error in this canvas example:

```
…
<div id="app">
  <canvas ref="draw"
/>
</div>
...
```

```
Vue.createApp({
  ...
  mounted() {
    this.ctx =
      this.$refs.draw.getContext("2d");
    let img =
      document.createElement("img");
    img.src = "fhlogo.png";
    this.ctx.drawImage(img,0,0);
  }
}).mount("#app");
```

Recap: Asynchronity

- Where is the error in this canvas example:

```
…
<div id="app">
  <canvas ref="draw"
/>
</div>
...
```

```
Vue.createApp({
  ...
  mounted() {
    this.ctx =
      this.$refs.draw.getContext("2d");
    let img =
      document.createElement("img");
    img.src = "fhlogo.png";
    this.ctx.drawImage(img,0,0);
  }
}).mount("#app");
```

! Recap: JavaScript Event Loop !

- JS is single threaded, not interruptable
- does not block on I/O operations but uses events
  - starts the I/O operation and continues with next line of code
  - if I/O operation is completed the system creates an event
- JavaScript Engine lets you register event handler functions
- JavaScript Engine stores all incoming events in a queue
  - first in first out (FIFO)

- when "idle": takes the first event available, executes the handler (w/o interruption)
- takes the next event available, executes the handler...

- a handler is never interrupted by a new event from the outside!

Recap: Solution I

- Loading images is an asynchronous operation!
  - use a handler function for load event

```
…
<div id="app">
  <canvas ref="draw"
/>
</div>
...
```

this/self: as the event handler will be called by the system, "this" is set to the calling object (global window object); notice the closure keeping the value of "self"

```
Vue.createApp({
  ...
  mounted() {
    this.ctx =
      this.$refs.draw.getContext("2d");
    let img =
      document.createElement("img");

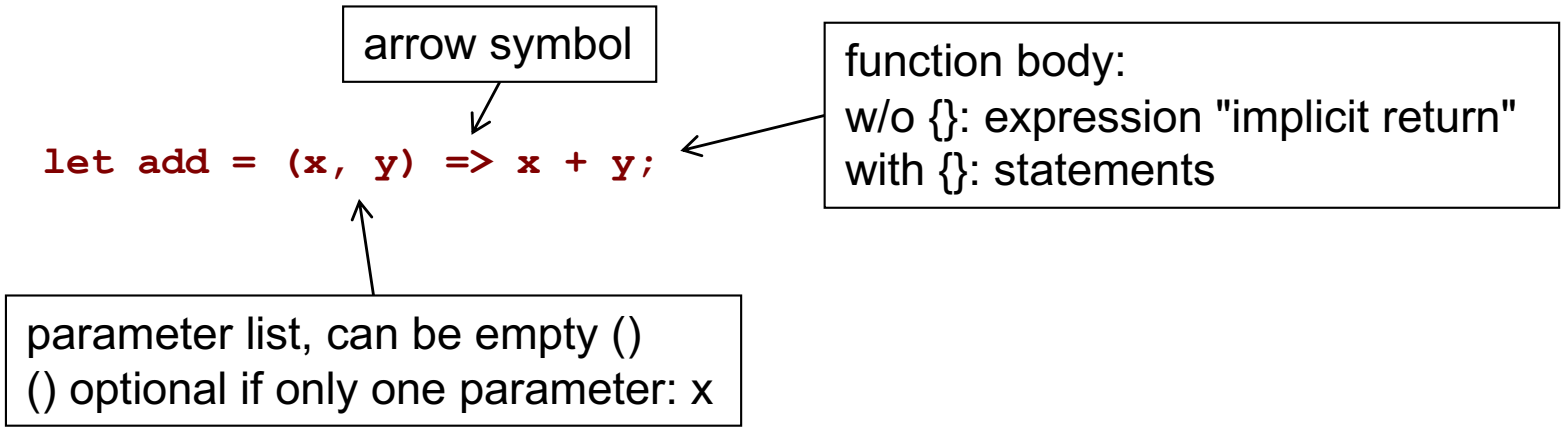
    let self = this;
    img.onload = function() {
      self.ctx.drawImage(img,0,0);
    }
    img.src = "fhlogo.png";

  }
}).mount("#app");
```

Recap: Arrow functions

- New syntax for anonymous functions

```
let add = function(x, y) { return x + y };  // old syntax
```



- Arrow functions use lexical context (= "surrounding code") for *this*
  - suitable whenever dynamic context is not needed
  - should not be used when declaring object methods

Recap: Solution II

- Loading images is an asynchronous operation!
  - use a handler function for load event

```
…
<div id="app">
  <canvas ref="draw"
/>
</div>
...
```

```
Vue.createApp({
  ...
  mounted() {
    this.ctx =
      this.$refs.draw.getContext("2d");
    let img =
      document.createElement("img");

    img.onload = () => {
      this.ctx.drawImage(img,0,0);
    }
    img.src = "fhlogo.png";

  }
}).mount("#app");
```

Recap: Vue Methods revisted:

- Don't use Arrow functions on "top level functions"
  - this will not work correctly
  - "this"-reference is managed by view, not possible with arrow functions

```
Vue.createApp({
  data: () => {return {}},     // WILL NOT WORK CORRECTLY
  methods: {
    drawBoard: () => {         // DO NOT DO THIS!
}
  },
  mounted: () => {             // NEVER, EVER!
    this.ctx =
      this.$refs.cvs.getContext("2d");
    let img = document.createElement("img");
    img.onload = () => { this.ctx.drawImage(0,0); };  // OK
    img.src = "fhlogo.jpg";
    this.drawBoard();
  }
}).mount("#app");
```

Recap: Web Worker

- Problem: cpu-intensive JavaScript blocks Browser
- Solution: Web Worker – Threading for JavaScript
  - JavaScript Code of a worker runs parallelly in background
  - Communication with main script via messaging

```
// main.js
let worker= new Worker("worker.js");
worker.onMessage= function(event) {
   alert("Worker sent: "+event.data);
};
function callWorker() {
   worker.postMessage("Hello Worker!");
}

//worker.js
this.onmessage = function(event) {
   postMessage("Hello Main Script, you sent: "+event.data);
}
```

Recap: Web Worker

- Worker are restricted
  - no access to dom elements of main script
  - restricted access to javascript APIs
  - Must be loaded from the same origin

- allowed APIs
  - postMessage / onmessage
  - set event listeners
  - XMLHttpRequest (send Ajax requests)
  - WebSockets
  - Timer
  - Location Objekt

- cancel a worker
  - worker.terminate()

Recap: Stopping a worker

- Why is this worker code erroneous?

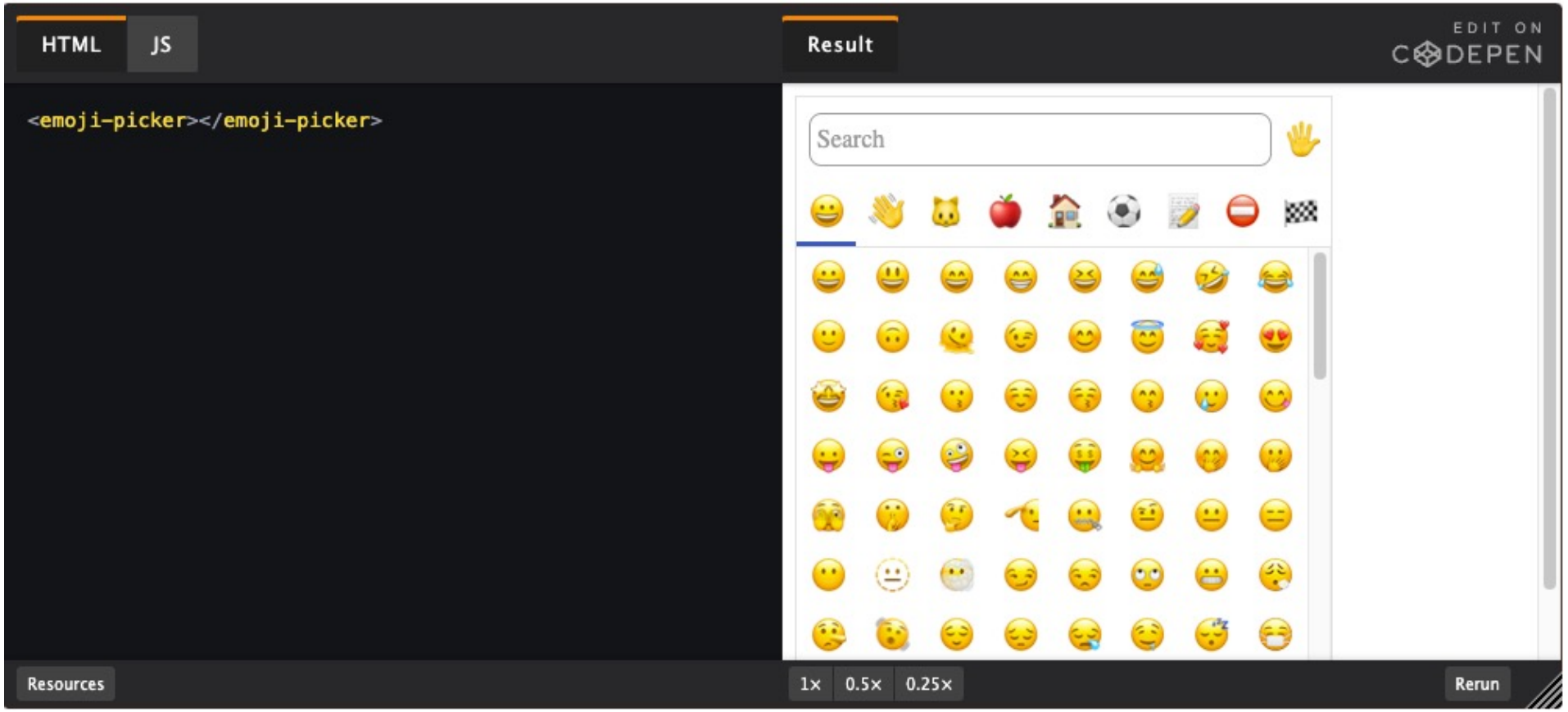
```
let running= false;
onmessage= function(event) {
  if (event.data==="start") {
    running= true;
    run();
  } else if (event.data==="stop") {
    running= false;
  }
}

function run() {
  while (running){ ... }
}
```

Web Components

- Several Specifications/APIs to enable developers to extend HTML
  - Custom Elements
  - Shadow DOM
  - Templates

Web Components: Example

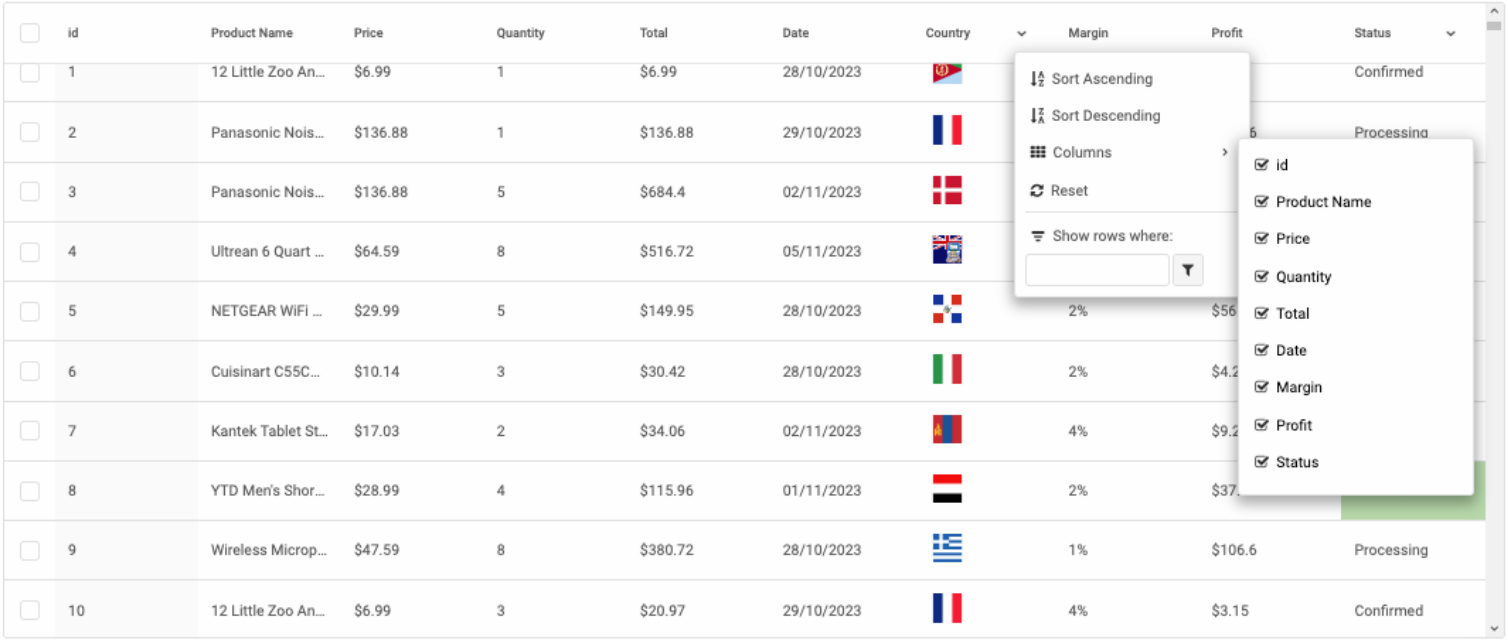


https://github.com/nolanlawson/emoji-picker-element

Web Components: Examples

Table Component

Table with 100,000 records. Our Table web component can be used to wrap or replace standard Tables and add different styles, hover effects, sorting by one or multiple columns, filtering, grouping, tree mode, add, remove and update rows.




```
<body class="viewport">
    <div class="demo-description">
        <h1>Table Component</h1>
        Table with 100,000 records. Our Table web component can be used to wrap or replace standard Tables and add different styles, hover effects,
        sorting by one or multiple columns, filtering, grouping, tree mode, add, remove and update rows. <br/>
        <br/>
    </div>
    <smart-table column-menu filtering sortable virtualization id="table"></smart-table>

    <!-- scripts -->
    <script type="module" src="../../source/modules/smart.table.js"></script>
    <script type="module" src="index.js"></script>

</body>
```

https://www.htmlelements.com/demos/table/overview/

Custom Elements

- web developers can define new types of HTML elements
  - via class inheritance
  - define new elements (extend standard class HTMLElement)
  - extend existing elements (extend existing element class)

- API function: `customElements.define(<name>,<class>)`

  - names must contain a dash (-)
    - <my-element>, <x-button>, …

```
class MyElement extends HTMLElement {…}
customElements.define("my-element", MyElement);
```

Use / Instantiate custom tag

- Declare

```
<my-element></my-element>
```

- Create DOM Node with standard **`createElement()`**

```
let myElement= document.createElement("my-element");
document.body.appendChild(myElement);
```

- with new operator

```
let myElement= new MyElement();
document.body.appendChild(myElement);
```

- custom Elements can be declared / created before their definition is registered with customElements.define()
  - –until they are upgraded thy are called UNRESOLVED elements
  - –new elements without dash (eg <myelement>) are UNKOWN elements and are ignored

Extending existing elements

- etxend native element

```
class MyButton extends HTMLButton { … }
customElements.define("my-button", MyButton, {extends: "button"});
```

- extend custom element

```
class MyExtendedElement extends MyElement { … }
customElements.define("my-xelement",
  MyExtendedElement, {extends: "my-element"});
```

- Usage declaratively

```
<button is="my-button"></button>
```

- Usage creating DOM Node

```
var myButton= document.createElement("button","my-button");
document.body.appendChild(myButton);
```

- Usage with new operator

```
let myButton= new MyButton();
document.body.appendChild(myButton);
```

"Public API" Methods

- "normal" methods can be added to the element class to create a "public API" for the element (like video.play() etc.)

```
class MyElement extends HTMLElement {
  test() { console.log("Test called"); }
}


let e = new MyElement();
e.test();
```

Element Lifecycle Methods

- certain methods are called automatically in the lifetime of a web component:
  - constructor
    - called on instance creation
  - connectedCallback
    - called when element is added to the DOM
  - disconnectedCallback
    - called when element is removed from DOM
  - attributeChangedCallback
    - called whenever an attribute listed in static getter "observedAttributes()" changes

```
class MyElement extends HTMLElement {
  static get observedAttributes() { return ["foo", "bar"]; }

  attributeChangedCallback(attr, oldVal, newVal) {
    switch(attr) {
      case "foo": // do something with "foo" attribute
      case "bar": // do something with "bar" attribute
    }
  }
}
```

Attributes vs Properties

- Attributes
  - set declaratively via HTML `<my-element count="13">`
  - API (inherited from base class Element):
    - `setAttribute(name, val)`
    - `getAttribute(name)`
    - `removeAttribute(name)`
  - should be declared in `static get observedAttributes()`
  - `attributeChangedCallback()`

- Properties
  - changed imperatively via JavaScript `e.count = 13;`
  - getter / setter

- KEEP THEM IN SYNC!
  - each Attribute should have a corresponding Property
    - `e.count = 13` -> `e.getAttribute("count")` ... 13 // use setter to sync
    - `e.setAttribute("count", 14)` -> `e.count` ... 14 // attributeChangedCallback ...
  - Exception: high frequency properties like video.currentTime

Sync Attributes & Properties

```
class MyElement extends HTMLElement {
  #foo;                                    // private field

  get foo() { return this.#foo; }          // public getter
  set foo(val) {                           // public setter
    this.#foo = val;
    this.setAttribute("foo", val);
  }

  static get observedAttributes() { return ["foo"]; }

  attributeChangedCallback(attr, oldVal, newVal) {
    switch(attr) {
      case "foo": this.#foo = newVal;
    }
  }
}
```

```
<my-element id"my" foo="ABC" /> ⟶ document.querySelector("#my").foo // ABC
```

Add Markup

- add markup via livecycle callbacks
  - typically on connectedCallback()

```
class MyElement extends HTMLElement {
  connectedCallback() {
    this.innerHTML = "This is the content of my Element";
  };
}
```

- Problem: content of custom elements is part of regular DOM – stylesheets / JavaScript might interfere, ids might overlap, …

Shadow DOM

- Encapsulation of the Presentation via Shadow DOM!

- Element-API: **`attachShadow({mode: "open"});`**
  - should be created in constructor not in connectedCallback()
  - connectedCallback() can be called multiple times (if element is de- and re-attached)
  - open mode: inspectable (in dev tools) and interactable (**`this.shadowRoot`**)
  - closed mode not recommended in "normal" cases

```
class MyElement extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({mode: "open"});
  }
}
```

Shadow DOM - Slots

- if a shadow DOM is created, the browser will render it instead of the element content
- `<slot>` (Element used in shadow DOM):
  access content of the custom element
  - can contain default value
  - can be named (see next slide)
  - "unnamed" slot is filled with all top-level child-nodes that do not have a slot attribute

```
<my-element>Testname</my-element>
[…]

class MyElement extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({mode: "open"});
  }
  connectedCallback() {
    this.shadowRoot.innerHTML = "Hello <slot>Default Text</slot>!";
  };
}
// Output: Hello Testname!
```

Shadow DOM – Named Slots

- slots can be identified with the Attribute **`name`**
- slot content is defined in custom element via the Attribute **`slot`**

```
[…]
// definition of custom element "element-details"
this.shadowRoot.innerHTML = `
<div>
  <h3>&lt;<slot name="element-name">NEED NAME</slot>&gt;</h3>
  <p><slot name="description">NEED DESCRIPTION</slot></p>
  <slot name="attributes"><p>No Attributes</p></slot>
</div>`;
[…]

<element-details>
  <span slot="element-name">h1</span>
  <span slot="description">Top level heading</span>
  <dl slot="attributes">
    <dt>id</dt><dd>The element id</dd>
    <dt>class</dt><dd>CSS classes</dd>
  </dl>
</element-details>
```

Shadow DOM Style

- Shadow DOM can be used like the standard document object
  - `this.shadowRoot.querySelector()`, ...

- Can have its own styles
  - internal style tag `<style>` or
  - external `<link rel="stylesheet" ...>`

- Additional selector `:host`
  - to style the custom element itself

  - custom elements use display: inline by default
  - to display them as block element (with width and height):

```
:host {
  display: block;
}
```

Templates

- the <template> tag allows to define parts of "inactive HTML", which means
  - resources (img, video) will not be loaded
  - scripts will not be executed
  - style will not take effect
- only cloning with importNode() & appending the clone to a DOM node brings its contents to life
- Example: use template for shadow DOM:

```
<template id="mytemplate">
<style>p { color: green }</style>
<p>Hello <span><slot></slot></span> from template!</p>
</template>

[…]
connectedCallback() {
  const template = document.querySelector("#mytemplate");
  const clone = document.importNode(template.content, true); // true: deep copy
  this.shadowRoot.appendChild(clone);
}

[…]
```

Vue Components

- Vue.js supports the creation of custom elements
  - component name contains a dash "-" (DOM compatible)
  - no shadow dom support, but you can use slots-elements in the template
  - only usable inside a root Vue instance, definition via
    - .component() or
    - option **`components:`**
  - "template" option for html contents of component

```
const app = Vue.createApp({});

app.component("my-counter", {
  data() { return { count: 0 }},
  template: "<div @click='count++'>{{ count }}</div>",
});

app.mount("#app");
```

.js File

```
...
<div id="app">
  <my-counter></my-counter>
</div>
...
```

.html File

x-templates

- Vue supports "x-templates"
  - use element-id as value for option template

```
const app = Vue.createApp({});

app.component("my-counter", {
  data() { return { count: 0} },
  template: "#mytemplate",
});

app.mount("#app");
```

.js File

```
<div id="app">
  <my-counter></my-counter>
</div>
...
<script type="x-template" id="mytemplate">
  <div @click="count++">{{ count }}</div>
</script>
```

.html File

Props

- to pass data to child components via attributes
  - accessible like reactive data elements
    - directly in templates, with this.<name> in methods
  - should be used read-only

```
const app = Vue.createApp({});

app.component("blog-post", {
  props: ["title"],
  template: "#mytemplate",
});

app.mount("#app");
```

.js File

```
<div id="#app">
  <blog-post title="Test Entry"></blog-post>
  ...
</div>

<script type="x-template" id="mytemplate">
  <h3>{{ title }}</h3>
  ...
</script>
```

.html File

Props

- Can also be used with v-bind, v-for

```
const app = Vue.createApp({
  data() { return {
    posts:[{title: "A"}, {title: "B"}, {title: "C"}]
  }}
});
app.component("blog-post", {
  props: ["title"],
  template: "#mytemplate",
});
app.mount("#app");
```

.js File

```
<div id="app">
  <blog-post v-for="post in posts" v-bind:title="post.title">
  </blog-post>
</div>

<script type="x-template" id="mytemplate">
  <h3>{{ title }}</h3>
  ...
</script>
```

.html File

Events

- send events to parent element
- $emit method
- event name can be self-chosen, should be declared in option 'emits'

```
const app = Vue.createApp({
  methods: {
    postHandler() {alert("Post Event"); }
}});
app.component("blog-post", {
  props: ["title"],
  emits: ["post-event"],
  template: "#mytemplate",
});
app.mount("#app");
```

.js File

```
<div id="app">
  <blog-post title="Test Entry"
    @post-event="postHandler"></blog-post>
</div>

<script type="x-template" id="mytemplate">
<h3>{{ title }}</h3>
<button @click="$emit('post-event')">…</button>
</script>
```

.html File

Events

- Optionally $emit can pass on a parameter
- receive the value via $event

```
const app = Vue.createApp({
  methods: {
    postHandler(text) {alert(text); }
  }});
app.component("blog-post", {
  props: ["title"],
  emits: ["post-event"],
  template: "#mytemplate",
});
app.mount("#app")
```

.js File

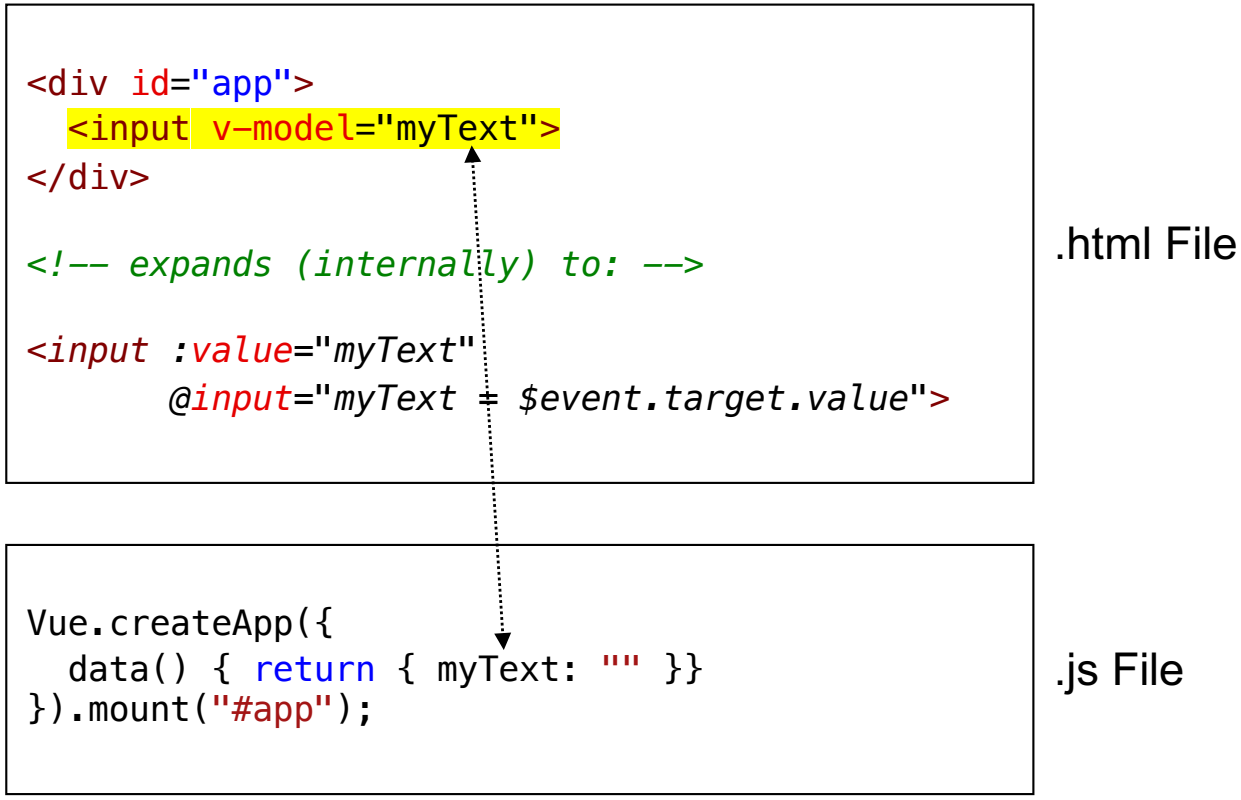
```
<div id="app">
  <blog-post title="Test Entry"
    @post-event="postHandler($event)"></blog-post>
</div>

<script type="x-template" id="mytemplate">
<h3>{{ title }}</h3>
<button @click="$emit('post-event','Hello World!')">…</button>
</script>
```

.html File

Recap v-model Direktive

- the directive v-model connects a reactive data element with an input field and provides "two-way binding" :



v-model and components

- if a component contains an input field, you can use the v-model directive for it; it will use by default
  - modelValue as prop
  - update:modelValue as event

```
<div id="app">
  <custom-input v-model="myText"></custom-input>
</div>

<!-- on a component v-model expands (internally) to: -->

<custom-input :modelValue="myText"
              @update:modelValue="newValue => myText = newValue"></custom-input>
```

.html File

- the component has to be implemented like this:

```
const app = Vue.createApp({
  data() { return { myText: ""}}
});
app.component("custom-input", {
  props: ["modelValue"],
  emits: ["update:modelValue"],
  template: `<input :value="modelValue"
                    @input="$emit('update:modelValue',$event.target.value)"/>`
});
app.mount("#app");
```

.js File

v-Model Arguments

- the default name "modelValue" can be changed with an argument to the directive v-model:

```
<div id="app">
  <custom-input v-model:title="aTitle"></custom-input>
</div>

<!-- v-model now expands (internally) to: -->

<custom-input :title="myText"
              @update:title="newValue => aTitle = newValue"></custom-input>
```

.html File

- the component has now to be implemented like this:

```
const app = Vue.createApp({
  data() { return { aTitle: ""}}
});
app.component("custom-input", {
  props: ["title"],
  emits: ["update:title"],
  template: `<input :value="title"
                    @input="$emit('update:title',$event.target.value)"/>`
});
app.mount("#app");
```

.js File

Multiple v-model bindings

- with v-model arguments, multiple bindings on a single component instance are possible
  - necessary if the component contains multiple input fields

```
<div id="app">
  <user-name v-model:firstName="first" v-model:lastName="last"></user-name>
</div>
```

.html File

- the component has now to be implemented like this:

```
const app = Vue.createApp({
  data() { return {
    first: "",
    last: "",
  }},
});
app.component("user-name", {
  props: ["firstName", "lastName"],
  emits: ["update:firstName", "update:lastName"],
  template: `<input :value="firstName"
                    @input="$emit('update:firstName',$event.target.value)">
             <input :value="lastName"
                    @input="$emit('update:lastName',$event.target.value)">`
});
app.mount("#app");
```

.js File